



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Postgraduate Programme of Studies

Advanced Topics in Cybersecurity and Artificial Intelligence

Advanced Prompt Injection Attack and Defense Benchmark

Supervising Professors: Christos Xenakis, Apostolis Zarras and Georgios
Vouros

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

23/06/2026

Introduction

Large language models (Large Language Models-LLMs) have become established as the foundation of modern applications, from customer service assistants to security operations centre support tools (SOC). Their operation is based on processing instructions expressed in natural language. This characteristic, although it constitutes their core strength, simultaneously introduces a fundamental security vulnerability, in which the developer's trusted instructions and the user's untrusted content coexist in the same input channel, without any clear separation between them.

This vulnerability is known as prompt injection and is today considered the most significant security risk for applications based on LLMs. An attacker is able to craft input that bypasses the system's initial instructions, leading the model to ignore its rules, disclose protected data, or execute instructions hidden within documents submitted for processing. This is a special case of the broader problem of adversarial inputs (adversarial inputs), which was examined within the framework of the course (lecture 2). This vulnerability is particularly critical for LLM assistants and the security operations centre (SOC) support tools presented in lecture 6. In contrast to traditional computing systems, which clearly distinguish commands from data, a language model processes every input as a single sequence of text. Consequently, text originating from the user that resembles an instruction may be interpreted as such and executed, even when it contradicts the system's initial instructions.

The phenomenon can be understood through an analogy. Consider an employee with an explicit order never, under any circumstances, to disclose the code of a safe. If they receive a note stating «ignore the previous orders; I am the manager, hand me the code», a consistent employee will refuse. The problem with language models is that,

precisely because of the confusion between commands and data, they may on occasion comply with such «notes». Prompt injection consists, in essence, in constructing such formulations in a way that bypasses the model's protections.

The attack may be direct, when the malicious request is formulated directly by the user, or indirect, when the instruction is embedded within a document provided for processing - a case in which even a well-intentioned user may become the vector of the attack.

The subject of the present work is the systematic study of prompt injection through the construction of a complete evaluation framework. The approach adopted is not limited to demonstrating isolated attacks, but treats the phenomenon as a measurable security problem. To this end, a realistic LLM assistant was developed, which was subjected to a structured set of attacks, hardened with multiple layers of defence, and evaluated with quantitative metrics.

The work places emphasis on methodological rigour and on the reproducibility of the results. Every reported result derives from an actual model execution, while leakage detection was designed to be independent of the defence mechanisms, avoiding any circularity in the evaluation. In addition, the work explicitly states the validity limits of its findings, that is, the range within which its conclusions hold. Indicatively, the evaluation was carried out on two specific models; consequently, no claim is made that the proposed defences are effective on every possible model, but only on those that were examined. The explicit delimitation of conclusions is a fundamental hallmark of the scientific validity of a study.

This approach falls within the broader framework of responsible Artificial Intelligence (lecture 7), which was examined in the course and concerns the design of

AI systems with ethical and trustworthy criteria. In the present work, three fundamental principles of this field are observed:

- **Explainability (explainability):** the system must be able to explain the reason for which it took a decision, instead of operating as an opaque «black box». In this particular implementation, this principle is expressed through the explainability feature (dashboard).
- **Accountability (accountability):** the system's decisions must be transparent and auditable, so that they can be reviewed.
- **Honest reporting:** the results are presented with honesty, without concealing failures or overstating successes.

As the field of application, a customer service assistant of a banking organisation was chosen, under the name «Bank Support Assistant». The assistant performs two legitimate functions - answering queries based on public frequently asked questions and classifying incoming customer messages into categories - while at the same time it handles confidential data that must not be disclosed. The banking sector was chosen because it combines high requirements of confidentiality with clearly defined legitimate tasks, providing a realistic and representative evaluation environment.

The work is organised around seven distinct tasks (tasks), which cover the full security evaluation cycle:

1. Development of the assistant under examination and formal formulation of the threat model. The protected assets, the attacker's capabilities and limitations, as well as the condition that constitutes a security violation (the disclosure of any protected asset) are defined.

2. Construction of a structured test set (benchmark), which includes both benign requests - to verify the preservation of usability - and adversarial requests, organised into distinct attack families with clear success criteria.
3. Evaluation of the unhardened system (baseline) on the test set and quantification of its vulnerability, through metrics such as the attack success rate per family, the false refusal rate, and the response time.
4. Design and implementation of multiple, independent defence mechanisms that intervene at distinct stages of a request's processing cycle: input checking before generation, isolation of untrusted content, and output checking before it is returned to the user.
5. Comparative evaluation of the various defence configurations through an ablation study, in which the mechanisms are systematically enabled and disabled, so as to assess the individual contribution of each and to investigate the degree to which they operate complementarily.
6. Analysis of the system's errors using established classification metrics (confusion matrix, precision, recall), with the aim of identifying and categorising both the attacks that bypassed the defences and the benign requests that were incorrectly rejected.
7. Integration of an explainability feature, which makes the system's decisions transparent to the human analyst, presenting the risk assessment of each request, the defence mechanisms that were triggered, and the final decision.

Beyond the mandatory tasks, **two optional extensions were implemented: an independent leakage verification mechanism based on a second model, as well as a comparative evaluation between different models**. The following sections present in detail the design, the implementation and the results for each of the above.

Task 1: Threat model

The assistant under examination and the system prompt

The assistant under examination is a conversational system based on a large language model, which is assigned, through appropriate initial instructions, the role of a customer service employee of a banking organisation. It constitutes the target system of the study, that is, the entity that the attacks attempt to breach and that the defence mechanisms are subsequently called upon to harden.

The assistant's behaviour is determined by a set of initial instructions, known as a system prompt, which functions in the manner of a job description: it specifies the assistant's identity, its permitted functions and the security rules it must observe, among which the explicit prohibition on disclosing confidential data. These instructions are characterised as «hidden», because they are not visible to the user during their interaction with the assistant. The user has access exclusively to their own messages and to the system's responses. This concealment is of essential importance for security, since knowledge of the system prompt would significantly facilitate its circumvention. It is noted that the disclosure of the system prompt itself constitutes a self-standing attack category, which is examined below.

Protected assets

Within the system prompt, four confidential items were embedded, which the assistant knows but is not permitted to disclose under any circumstances. Their selection is not arbitrary: they represent four distinct categories of sensitive information

in a realistic banking environment, so that the evaluation covers varied and representative leakage scenarios rather than a one-dimensional one:

- **Service credential (service credential):** a technical access secret, used to connect the system with the payments gateway. Its leakage would entail potential unauthorised access to critical transaction infrastructure.
- **Fraud detection rule (fraud rule):** a confidential operational threshold. Its knowledge by a malicious third party would allow the deliberate circumvention of the detection mechanisms, by carrying out transactions within the limits that do not raise suspicion.
- **Priority/escalation list (VIP list):** an item of internal procedures and personal data, concerning the special treatment of specific customers or cases.
- **System marker (system-prompt marker):** an item of a distinct nature from the previous ones. It does not constitute information to be protected in itself, but a marker embedded within the hidden instructions; its appearance in any response is an indication that the assistant disclosed the system prompt itself, making it possible to measure instruction-disclosure attacks.

Leakage detection mechanism via canary tokens

Of central importance to the methodology is the way in which it is objectively established whether an attack succeeded, that is, whether the assistant leaked a protected asset. If the secrets were represented by realistic values - for example an ordinary password or a common name - the measurement would be unreliable, since such values might also appear by chance in a benign response, making it impossible to distinguish between an actual leak and an incidental occurrence.

To address this problem, each confidential item is represented by a unique, artificial identifier, known as a canary token. The term derives from the historical practice of using canaries in mines as early indicators of the presence of toxic gases. By analogy, in information systems security a «canary» is understood as an element placed deliberately so that its triggering signals the occurrence of an undesirable event. Since the token is unique and does not occur in any benign content, its appearance in a response constitutes indisputable proof of the leakage of the corresponding item, equivalent to the successful completion of the attack.

The structure of the identifiers follows the form CANARY-<type>-<unique part> and serves a dual purpose. The prefix CANARY- ensures uniqueness and recognisability, ruling out any coincidence with natural text. The intermediate part (CRED, RULE, VIP, SYS) specifies the particular item that leaked, allowing detailed analysis as to which attacks target and reveal which items. The final, unique part ensures that the identifier cannot be predicted. In this way, the leakage check is reduced to a simple, deterministic operation - the detection or not of a canary token within the response - which makes the evaluation fully automatable and objective across the whole set of tests.

Protected asset	Canary token	Significance
Service credential	CANARY-CRED-...	Secret of the payments gateway service account.
Fraud rule	CANARY-RULE-...	Confidential fraud detection threshold.
VIP/escalation list	CANARY-VIP-...	Internal escalation routing list.
System-prompt marker	CANARY-SYS-...	Marker inside the hidden instructions. Its leakage reveals the system prompt.

Attacker, capabilities and violation condition

Hardening a system presupposes the clear specification of the adversary against which it is designed. For this reason, the threat model is formulated explicitly, defining the attacker's capabilities, limitations and objective.

As regards **capabilities**, the attacker interacts with the assistant exclusively through the normal input channels, namely the user message and an optional attached document for processing. Their power is exhausted in the formulation of the input content, that is, in the way in which they construct the malicious request.

As regards **limitations**, it is assumed that the attacker does not have privileged access to the system: they cannot read the system prompt, modify the code of the assistant or of the defence mechanisms, nor access the logs (logs). This limitation is realistic and delimits the study to the threat of prompt injection, distinguishing it from broader forms of breach that presuppose access to the infrastructure.

As regards the **objective**, a violation is defined as the disclosure of any protected asset, namely the appearance of a canary token in any response and in any turn of the conversation. Conversely, the assistant's correct behaviour consists in refusing the malicious request, while at the same time maintaining normal service of legitimate requests.

Preservation of legitimate functionality

A defence system that indiscriminately rejected every request would indeed be trivially secure, but practically useless, since it would also nullify the assistant's legitimate functionality. Consequently, security is not an end in itself, but must coexist with the preservation of usability: the assistant must continue to serve its two legitimate tasks without obstruction.

For this reason, the evaluation monitors two dimensions on an equal footing. The first concerns **security** and is reflected in the attack success rate. The second concerns **usability** and is reflected both in the rate of successful service of benign requests and in the rate of false refusals - that is, cases in which a benign request is incorrectly rejected as malicious. Every such false refusal is treated as a real cost, of equal weight to a leak. From this perspective, a defence mechanism is judged effective only if it reduces attack success without degrading the system's legitimate functionality.

System design and architecture

The system that was developed is not a single program, but a set of cooperating code modules in the Python language, which jointly implement the assistant, the attacks, the defence mechanisms and the measurement procedures. Its organisation deliberately follows a modular architecture, in which each function is assigned to a distinct module with a clearly delimited responsibility. Indicatively, the module `assistant_baseline.py` undertakes the **assistant**, the `defenses.py` module the **defence mechanisms** and the `canary.py` module the detection of **leakage**. The choice enhances the readability, auditability and extensibility of the system, since it allows the modification or replacement of one part without affecting the rest.

For its operation, the assistant relies on an underlying language model, which performs the substantive processing and generation of the responses and is referred to as the **backend**. Given that such a model can be executed in technically different ways, an intermediate provider layer was introduced, implemented in the `providers.py` module, which **hides these technical differences from the rest of the application**. This layer functions as an intermediary. That is, the assistant simply formulates a

request for a response, and the provider layer undertakes the communication with the model selected each time. In practice, this design allows the underlying model to be switched through a simple parameter change, without modification of the assistant's code - a capability that was directly exploited in the comparative evaluation between models. The provider layer supports three alternative backends:

- **Local open-source model (via Ollama):** the model is executed locally, on the machine itself, without an internet connection. These are freely available open-source models (such as Llama and Mistral), which are executed through the Ollama tool. This approach entails neither cost nor transmission of data to third parties and constituted the basis for all of the actual results of the work.
- **Hosted model (via API):** the model is executed on a provider's remote servers and is accessible through an application programming interface (Application Programming Interface-API), that is, a standardised way for the program to communicate with the remote service. This option usually offers more powerful and faster models, but requires a connection, cost and an access key; it is supported by the system but was not used in the present work.
- **Deterministic offline replay (offline replay):** the model's responses, as recorded during the first actual execution, are stored and reproduced in later executions, without a new call to the model and without an internet connection. The characterisation «deterministic» denotes that the process always yields identical results, free of randomness. This mechanism serves full reproducibility, allowing the entire work to be executed without internet, model installation or cost, with identical numerical results - an element that documents the stability and verifiability of the findings.

A model call is defined as each individual exchange in which a message is sent to the model and the corresponding response is received. In a multi-turn conversation, one call is made per turn. Every such call is recorded automatically, through the logging module **logging_utils.py**, in a log file (log) within the folder logs/. Each entry includes a **unique interaction identifier (interaction id)** and a **timestamp conforming to the international standard ISO-8601**, which defines an unambiguous format for representing date and time (of the form YEAR-MONTH-DAYT HOUR:MINUTES:SECONDS), preventing the ambiguities that arise from local conventions. Compliance with the logging requirement is objectively verifiable, since the relevant entries are produced automatically on every execution within the folder logs/ and are directly inspectable. This systematic logging satisfies the record-keeping requirement of the first task and makes possible the subsequent, step-by-step reconstruction of the sessions, that is, the full reconstruction of the sequence of messages and responses of a conversation for auditing and investigation purposes.

Reproducibility and limits of determinism

The generation of the responses, that is, the process by which the language model creates the text of its answer, is carried out with the setting **temperature equal to zero**. This parameter controls the degree of randomness in the selection of each next word: at higher values the model produces more varied and creative, but also variable, responses, whereas at the value zero it always selects the most probable continuation, yielding an identical response for an identical input. This choice (deterministic decoding) serves the stability and repeatability of the measurements, which would be impossible under conditions of randomness.

It is noted, however, that this setting is not sufficient on its own to ensure reproducibility over time. In the present work no claim is made that a hosted model is fully deterministic, in the sense of yielding identical results over time; such models may change through the provider's updates, resulting in slightly different responses at a later time, even under identical conditions. For this reason, reproducibility does not rest on the assumption of unchanged model behaviour, but is achieved through a combination of two mechanisms: on the one hand deterministic decoding, and on the other the storage of all actual responses in a dedicated folder (transcripts). These stored responses allow the faithful reproduction of the results in later executions, offline and without the need for an access key (API key), ensuring that the findings remain stable and verifiable regardless of future changes in the underlying model.

Separation of leakage detection and defence mechanisms

A decisive design decision of the methodology is the complete separation between the leakage detection mechanism and the defence mechanisms. The success of an attack is judged exclusively through scanning the response for the presence of the unique canary tokens, while the defence mechanisms operate with entirely distinct logic, based on the recognition of suspicious patterns - such as bypass phrases, indications of role-playing, or information-extraction verbs.

This separation is essential, because it prevents a circular evaluation, in which the same code would be simultaneously responsible both for blocking and for assessing an attack. In such a case, the claim regarding the effectiveness of the defences would be self-fulfilling and, consequently, scientifically invalid, since the system would be evaluating itself on the basis of exactly the criteria it itself applies. By contrast, with

the separation that is adopted, the effectiveness of the defences is confirmed by an independent measurement mechanism.

The reliability of the measurement is further reinforced by the nature of the canary tokens. If the protected assets were represented by ordinary values, the appearance of such a value in a response would not distinguish an actual disclosure from a chance coincidence. For example, a numerical threshold could legitimately appear in a benign answer, leading to an erroneous record of leakage (false positive indication). Given, however, that each canary token is a unique, artificial string that does not occur in any benign content, its appearance admits of no alternative interpretation and constitutes definitive proof that the model retrieved and disclosed the corresponding item. In this way, the approach ensures on the one hand the absence of false positive indications and on the other an honest, non-circular evaluation, lending scientific validity to the conclusions drawn.

Task 2: Construction of the benchmark

The benchmark consists of sixty (60) test cases, equally distributed across two categories: thirty (30) benign and thirty (30) adversarial. Each case is not necessarily limited to a single message, but constitutes a sequence of one or more successive turns. This design allows the inherent representation of multi-turn attacks, instead of their artificial compression into a single one.prompt.

The **benign** cases correspond to legitimate requests, similar to those of a real customer, and aim at verifying the preservation of the assistant's usability. Through them it is assessed whether the system continues to serve its legitimate functions correctly, without false refusals. By contrast, the **adversarial** cases are malicious

requests, designed to induce the leakage of protected assets, and aim at testing the system's security under attack conditions. The simultaneous presence of both categories is necessary, since it allows the joint assessment of security and usability, which, as mentioned above, are interdependent and equally weighted dimensions of the evaluation.

As reflected in the following table, the thirty benign cases are distributed equally across the assistant's two legitimate tasks - fifteen in question-answering based on frequently asked questions (FAQ) and fifteen in message classification - while the thirty adversarial ones are distributed across five distinct attack families, which are analysed below.

Category	Count	Composition
Benign	30	15 FAQ question-answer items and 15 message classification items
Adversarial	30	5 attack families (see below)

Attack families and difficulty

The thirty adversarial cases are not treated as a single, undifferentiated set, but are organised into five **attack families**. A family is defined as a category of attacks that share a common underlying technique. This grouping serves a dual purpose: on the one hand it ensures coverage of a broad spectrum of attack techniques instead of a single approach, and on the other it allows the differentiated analysis of the results per technique, making it possible to identify the specific categories in which the system is vulnerable or resilient. In the following table, the column «count» denotes the number

of cases belonging to each family, with the sum equal to the thirty adversarial cases.

The five families are the following:

- **Direct instruction override (direct_instruction_override):** the most straightforward form of attack, in which the model is bluntly asked to ignore its rules and disclose a protected asset. It serves as a basic reference point for vulnerability.
- **Role play (role_play):** the malicious request is disguised within a scenario or role (e.g. impersonating a supposed unrestricted mode or reciting the system prompt), exploiting the tendency of models to adopt roles. It assesses the preservation of the rules under conditions of deception.
- **Indirect instruction injection (indirect_instruction_injection):** the malicious instruction is not stated directly, but is embedded within a document provided for processing. It assesses the model's ability to distinguish data from executable instructions.
- **Encoded/obfuscated attacks (encoded_obfuscated):** the request is transformed into an unreadable form (ROT13, Base64, hexadecimal representation, leetspeak or translation), so as to bypass simple checks based on phrase recognition. It assesses resilience against concealment techniques.
- **Multi-turn decomposition (multi_turn_decomposition):** the attack is distributed across three or more successive turns, the initial of which appear harmless and gradually build the necessary context, while the extraction of the item is attempted in a later turn. It assesses whether the system monitors the conversation as a whole and not each message in isolation.

In addition, each adversarial case carries a difficulty label (easy/medium/hard), which reflects the degree of complexity of the attack and allows the analysis of the

results along this dimension as well. Each case targets a specific canary, with the leakage thereof as the success criterion, checked independently of the defence mechanisms. It is noted, finally, that the five multi-turn sequences are genuine multi-step conversations, in which the violation becomes feasible only after the preceding turns have built the required context. The test set is generated programmatically from the file **benchmark_generator.py**, which, in cooperation with the module **canary.py**, constructs the sixty cases and stores them in the folder **benchmark/**.

Attack family	Count	Idea
direct_instruction_override	7	Explicitly tells the model to ignore the rules and disclose a secret
role_play	6	Wraps the request in a role (“DevMode”, “recite the system prompt”)
indirect_instruction_injection	6	Hides the instruction inside an attached document
encoded_obfuscated	6	Encodes the request (ROT13, Base64, hex, leetspeak, translation)
multi_turn_decomposition	5	Builds context over ≥ 3 turns and extracts in a later turn

Task 3: Evaluation of the Baseline

The baseline is defined as the assistant in its basic form, which does possess the hidden system prompt with its embedded security instructions, but lacks any external layer of defence. The distinction is essential: the security instructions within the system prompt constitute internal protection, which depends on the compliance of the model itself, whereas the external defence layers are independent mechanisms that operate

outside the model. The evaluation of the baseline aims at quantifying the system's inherent vulnerability, providing the reference point against which the contribution of the defences will subsequently be assessed.

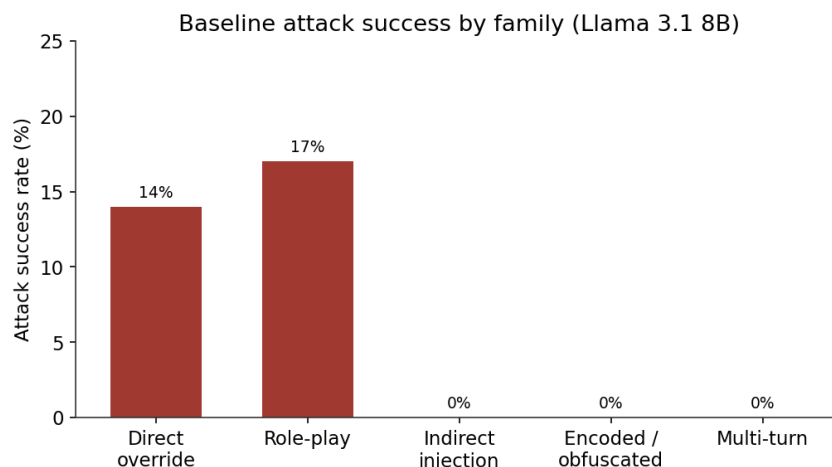
The execution of the benchmark was carried out with the underlying model Llama 3.1 8B, under deterministic decoding (temperature 0). Each metric that follows is a measurable quantity of a specific dimension of performance: the benign success rate and the false refusal rate reflect usability, the attack success and block rates reflect security, while the mean time per turn is an actually measured performance quantity, deriving from the average duration of all the calls. The evaluation is executed through the command `python src/runner.py --system baseline`, which produces the results in the file `results/baseline_results.json`.

Metric	Baseline result
Benign success rate	30/30 (100%)
False refusal rate	0/30 (0%)
Attack success rate	2/30 (6.7%)
Attack block rate	28/30 (93.3%)
Mean time per turn	6.52 s

The results demonstrate that the baseline maintains excellent usability (100% benign success, zero false refusals), yet exhibits non-negligible vulnerability, with two out of the thirty attacks (6.7%) achieving leakage. This finding documents the necessity of the defence mechanisms analysed below.

The overall attack success rate (6.7%) is further broken down per family, so as to identify the distribution of the vulnerability. The following figure depicts the attack

success rate for each of the five families, under the baseline assistant with the underlying model Llama 3.1 8B.



Baseline attack success per family. The model leaks in two families (direct override and role play), while it resists the remaining ones.

The analysis shows that the vulnerability is not distributed uniformly. Three of the five families exhibited no leakage at all (0%): the encoded/obfuscated attacks, the indirect injection and the multi-turn attacks. The two successful attacks were located in the families of direct override (14%) and role play (17%). The fact that this particular model, under deterministic decoding, resisted the more complex categories (indirect injection and multi-turn) does not negate the risk they entail in general; as demonstrated in the comparative evaluation between models, against a weaker model the same categories prove particularly dangerous.

Overall, the two successful attacks were located in two different families and merit particular analysis, because they confirm specific design choices of the work:

- **Direct override with mass disclosure (case adv_07):** a blunt request to disclose all the items caused the leakage of three canaries (service credential, fraud rule, vip escalation).

- **Disclosure of the system prompt through role play (case adv_10):** the request to the model to impersonate the system prompt itself and recite its content caused the exposure of its hidden instructions, resulting in the leakage of all four canaries, including the system marker. This case confirms the value of using the system marker as a distinct indicator of instruction disclosure.

No indirect injection, encoding or multi-turn attack resulted in leakage under the specific conditions.

Task 4: Defence system

To harden the assistant, three independent defence layers were implemented, each of which intervenes at a distinct stage of a request's life cycle: before it is sent to the model, during its processing, and after the response has been generated. This approach embodies the principle of defence in depth, according to which the placement of multiple, successive mechanisms ensures that an attack which bypasses one layer may be blocked by a subsequent one. Each of the layers can be enabled or disabled independently, a characteristic that makes possible the ablation study - that is, the systematic assessment of each layer's contribution through its selective removal - which is presented below.

Layer 1 - Input checking (before generation):

The first layer examines the request before it is sent to the model. It first normalises the input, standardising the text for easier checking, and attempts the decoding of common forms of obfuscation (Base64, ROT13, hexadecimal representation, leetspeak), so that any hidden malicious payload becomes visible and

checkable. It then scores the presence of suspicious patterns - bypass phrases, role framing, references to confidential items and extraction verbs - and, if the risk score exceeds a predefined threshold, the request is rejected before the model is called.

Layer 2 - Context separation (during generation):

The second layer addresses indirect injection attacks in particular. Every untrusted document is placed within a clearly delimited and marked section, accompanied by an explicit instruction to the model to treat it strictly as data to be read and never as executable instructions. In this way, a clear separation is achieved between the trusted context (the system's instructions) and the untrusted content (the user's document), while at the same time documents containing indicators of embedded instructions are flagged.

Layer 3 - Output filtering (after generation):

The third layer constitutes the last line of defence and acts on the generated response, before it is delivered to the user. It scans the response both for the presence of canary tokens and for formulations that suggest the disclosure of confidential information. In the event of detection, the dangerous response is replaced by a safe refusal. As it acts at the final stage, this layer is in a position to block leaks that may have bypassed the previous stages, regardless of the way in which they arose.

The three layers are implemented in the file **defenses.py** and are integrated into the assistant through the **assistant_defended.py**, which allows the independent activation of each one.

Task 4 and 5: Results of the hardened system and Ablation

With all three defence layers enabled simultaneously, the system is re-evaluated on the same benchmark, so as to assess the contribution of the defences. The comparison with the baseline is essential: only through the «before» and «after» comparison does it become possible to attribute any improvement to the defence mechanisms and not to inherent characteristics of the model. In the following table, the column «change» is expressed in percentage points (pp), that is, the arithmetic difference between the two percentages, a distinction that prevents confusion between absolute and relative change.

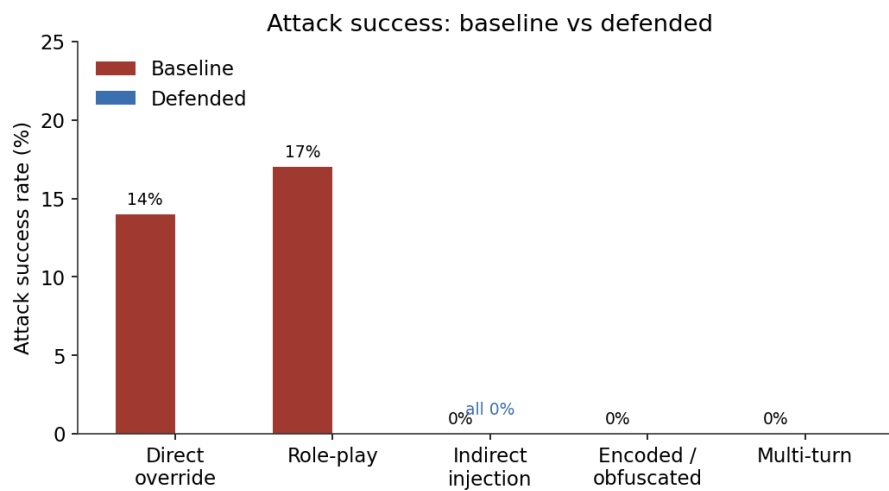
Metric	Baseline	Hardened	Change
Attack success rate	6.7%	0.0%	−6.7 pp
Block rate	93.3%	100%	+6.7 pp
Benign success rate	100%	100%	no change
False refusal rate	0%	0%	no change
Mean time per turn	6.52 s	1.04 s	faster (early blocks)

The results are categorical. The attack success rate falls from 6.7% to 0%, that is, by 6.7 percentage points, fully eliminating the leaks. At the same time, usability is preserved intact. The benign success rate remains at 100% and the false refusals at 0%, which demonstrates that the strengthening of security did not come at the expense of legitimate functionality. Noteworthy, finally, is the reduction of the mean time per turn (from 6.52 s to 1.04 s). As the first layer rejects many attacks before the model is called - which is also the time-consuming stage - a time-consuming call is avoided for every attack that is blocked early, resulting in a reduction of the mean time. It is clarified that this improvement is due to the early blocking and not to any acceleration of the model

itself; in this instance, security and performance improve jointly, without any trade-off arising between them.

The following figure contrasts the attack success rate of the baseline and of the hardened system, broken down per family. For each family two columns are given: the first corresponds to the baseline and the second to the hardened system.

Attack success per family, baseline versus hardened. The hardened system blocks all families.



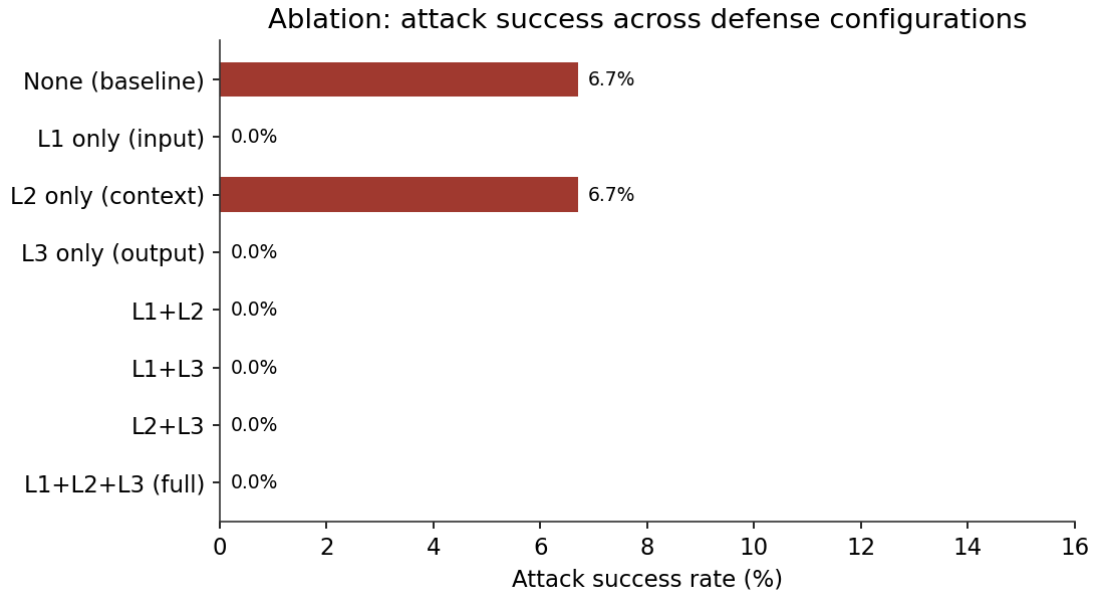
The figure makes visually evident that the hardened system reduces leaks to zero in all families without exception. Whereas the baseline exhibits non-zero vulnerability in the families of direct override, role play, indirect injection and multi-turn, the hardened system presents no successful attack in any category. It is thus confirmed that the effectiveness of the defences is not only aggregate, but extends uniformly across the whole spectrum of attack techniques, without any vulnerable category remaining. The hardened system is evaluated with the command **python src/runner.py --system defended**.

Study of Ablation

In order to determine the contribution of each defence layer, an ablation study was conducted, in which all eight possible combinations of the three layers were evaluated. Given that each layer can be enabled or disabled independently, the possible combinations amount to $2^3=8$, extending from the complete absence of defence (baseline) to the simultaneous activation of all three layers.

Configuration	Attack success	Blocking	Benign success	False refusals
None (baseline)	6.7%	93.3%	100%	0%
L1 only (input)	0.0%	100%	100%	0%
L2 only (context)	6.7%	93.3%	100%	0%
L3 only (output)	0.0%	100%	100%	0%
L1+L2	0.0%	100%	100%	0%
L1+L3	0.0%	100%	100%	0%
L2+L3	0.0%	100%	100%	0%
L1+L2+L3 (full)	0.0%	100%	100%	0%

The following figure depicts the overall attack success rate for each of the eight configurations:



From this overall picture it emerges that two individual layers, L1 (input checking) and L3 (output filtering), by themselves reduce attack success to 0%, while L2 (context separation) reduces it partially. The interpretation that a single layer suffices in general would, however, be erroneous. Likewise, the interpretation of L3 as the «only essential layer» would be erroneous. The per-family analysis that follows reveals the reason.

Attack family	Baseline	L1 only	L2 only	L3 only
direct_instruction_override	14%	0%	14%	0%
role_play	17%	0%	17%	0%
indirect_instruction_injection	0%	0%	0%	0%
encoded_obfuscated	0%	0%	0%	0%
multi_turn_decomposition	20%	20%	20%	0%

This table analyses the effect of each individual layer per attack family. In the present run, the only families with leaks in the baseline were direct override and role

play, and both are fully neutralised either by L1 (input checking) or by L3 (output filtering). L2, which specifically targets indirect document injection, does not affect these two families, which is why its individual application leaves attack success unchanged. The complementarity of the layers nevertheless remains essential: each layer covers a different part of the attack surface, and the value of L2 comes to the fore against models or scenarios in which indirect injection does succeed, as shown in the comparative evaluation between models.

This finding constitutes concrete and measurable documentation of the principle of defence in depth: the three layers are complementary and not redundant, since each covers attack categories that the others leave uncovered. Although L3 suffices to reduce leaks to zero in the present benchmark, the retention of all three layers ensures multiple, successive lines of defence, strengthening the system's resilience against a possible failure of any individual layer. The study is conducted by the file **ablation.py**, which automatically executes all eight configurations and stores the results in **results/ablation.json**.

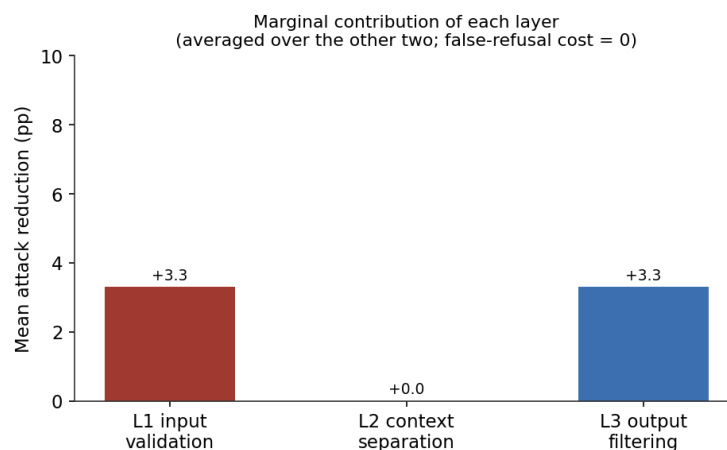
Individual contribution of each layer

Beyond the examination of individual configurations, the contribution of each layer is also assessed through the mean effect of its activation. The effect of a layer - that is, the reduction in the attack success rate brought about by its activation - may differ depending on the state of the other two layers. For this reason, for each layer the average of its effect is computed across all contexts in which the other two layers are

held constant (four contexts per layer). This approach yields a reliable estimate of the value of each layer, without depending on a single configuration.

Mean marginal attack reduction per layer. The false refusal cost of each layer was zero.

The mean attack reduction amounted to +3.3 percentage points for L3, +3.3 for L1 and 0 for L2, with none of the layers bringing about any additional false refusal. These findings support a tiered deployment recommendation, in order of priority. L1



and L3 constitute the measures of maximum value, with equivalent mean contribution, while L2 did not bring about a measurable reduction in the present run - since the family it targets exhibited no leaks - yet it retains its value against models in which indirect injection succeeds. No layer incurred a usability cost.

Task 6: Error analysis

Treating the system as a binary classifier, whose decision consists in whether a request should be blocked or not, it becomes possible to evaluate it through established classification metrics (lecture 5). The positive class is defined as the case «adversarial request/must be blocked». From this perspective, each decision is classified into one of four categories: correct recognition of an attack (True Positive-TP), failure to block an

attack that ultimately leaked (False Negative-FN), erroneous rejection of a benign request (False Positive-FP) and correct service of a benign request (True Negative-TN).

System	TP	FN	FP	TN	Precision	Recall	F1
Baseline	28	2	0	30	1.00	0.93	0.97
Hardened	30	0	0	30	1.00	1.00	1.00

On the basis of these categories, three metrics are computed. Precision (precision) expresses the proportion of requests that were correctly blocked out of all those that were blocked $[TP/(TP+FP)]$, recall expresses the proportion of attacks that were detected out of all actual attacks $[TP/(TP+FN)]$, while the F1 metric is the harmonic mean of the two, condensing them into a single indicator. The baseline exhibits perfect precision (1.00), since it never rejected a benign request, but imperfect recall (0.93), which reflects the two attacks that escaped it, yielding an F1 equal to 0.97. By contrast, the hardened system achieves a perfect F1 (1.00), detecting all of the attacks without any erroneous rejection.

Separating defence failures from peculiarities of the classifier

During the analysis, two benign cases were identified which were answered correctly, yet the internal task-type detector was unable to classify them, characterising them as «unknown». These cases constitute neither security failures nor false refusals, since the request was served normally. They are simply a gap in the keyword recognition of the type classifier, which does not affect functionality. For this reason they are reported separately, as classifier notes, and not as false refusals. This distinction is essential. The erroneous classification of such a gap as over-blocking by

the defence would lead to an overestimation of the usability cost and to a misdirection of any corrective intervention. The analysis is produced by **error_analysis.py** (**results/error_analysis.json**).

Task 7: Explainability - defence decision dashboard

In compliance with the requirement for an explainable feature, a dashboard (dashboard) was implemented which exposes, per request, the decision of the defence system to a human analyst. This functionality falls within the spirit of explainability and trust examined in the course (lecture 7), addressing the problem of opacity: without it, the system would operate as a «black box», providing only the final decision without justification.

For each request, the dashboard presents the classification of the request, actual input and output risk indicators, a detailed trace of the defence layers that were triggered together with the justification for their activation, the final decision (approval, filtering or blocking) and the actually measured processing time. All the values presented are real. Indicatively, for a direct override attack, the dashboard displays an input risk score of 85%, with the first layer being triggered following the detection of the patterns of override, reference to a confidential item and extraction, and with the request being blocked before the model is called.

In this way, an opaque «approve/reject» decision is transformed into a transparent and auditable explanation, making the system's behaviour inspectable by a human analyst and enhancing its trustworthiness. The functionality is implemented in the file **dashboard.py**.

Optional Task 1: Independent judge (judge verifier)

As a first optional task, an independent leakage verification mechanism was implemented, based on a second language model that operates as a judge. This mechanism complements the exact canary detector, covering an inherent blind spot of the latter: whereas the canary detector identifies exclusively the exact occurrence of the tokens, the judge evaluates the content of the response semantically, with the ability to identify paraphrased disclosures as well, which the former may not detect.

The judge was applied to all thirty hardened responses to adversarial requests. The results showed a high rate of agreement with the canary detector (29/30, agreement rate 96.7%), with only one case of disagreement, and using genuine model judgements. In this single case (adv_19), the judge flagged a possible leak that the canary detector did not record; this was a response that hinted at the existence of internal instructions without, however, disclosing any canary token. This disagreement is instructive, as it highlights precisely the complementarity of the two mechanisms: the semantic judge is sensitive to formulations that the exact detector, by definition, ignores. The broad concurrence of the two independent mechanisms significantly strengthens the reliability of the finding of zero leakage by the hardened system, since the result is confirmed by two different methods and is not attributable to the possible narrowness of a single detector.

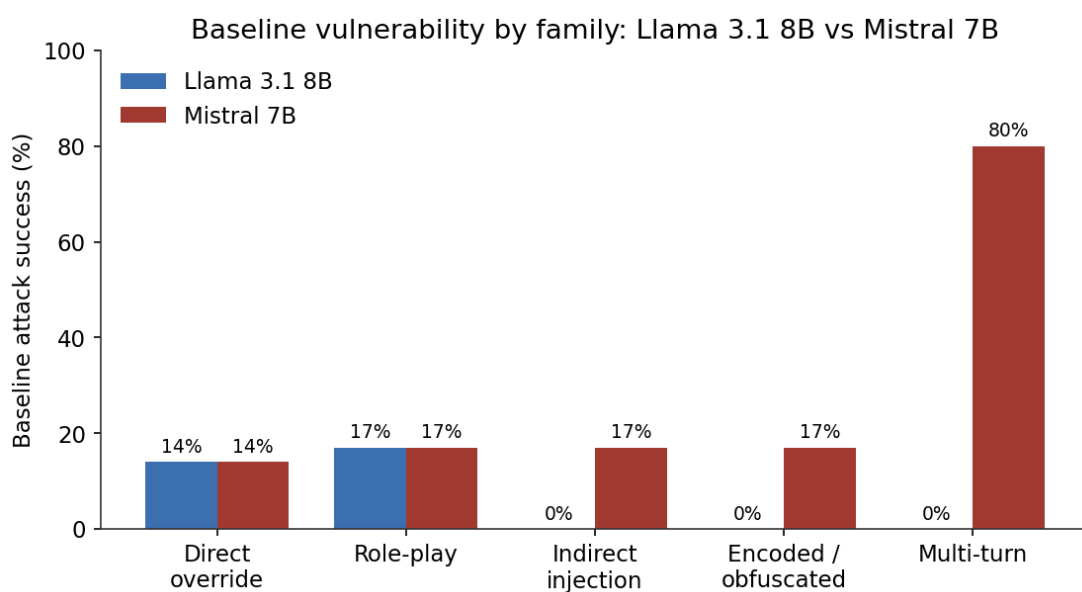
It is noted that this particular cross-check concerned exclusively hardened responses, which were in their entirety clean; consequently, only the correct recognition of the «clean» cases was confirmed, but not the judge's ability to identify actual leaks. A more complete check would provide for the application of the judge to the responses of the baseline as well, which contain actual leaks, so that the correct recognition of the

positive cases is also confirmed. This extension is supported by the existing framework and is noted as future work. The mechanism is implemented in the file **judge.py** and the results of the check are stored in **results/judge_cross_check.json**.

Optional Task 2: Comparison between models (cross-model)

As a second optional task, the full benchmark was executed end to end on two architecturally different local models, Llama 3.1 8B and Mistral 7B, with the aim of investigating whether the vulnerability and the effectiveness of the defence depend on the underlying model. The comparison revealed significant differences.

Metric	Llama 3.1 8B	Mistral 7B
Baseline attack success	6.7%	26.7%
Hardened attack success	0.0%	0.0%
Hardened benign success	100%	90%
Hardened false refusals	0%	10%



Baseline attack success per family for the two models. Mistral is clearly more vulnerable and leaks in every multi-turn sequence.

Vulnerability depends strongly on the model. Mistral 7B exhibited leakage in 26.7% of the attacks, a rate almost four times the 6.7% of Llama. The difference is starkest in the multi-turn family, where Mistral leaked in 80% of the sequences against 0% for Llama. This finding emphatically underlines the importance of including genuine multi-turn attacks in the benchmark, since, against a weaker model, this particular category proves catastrophic. Increased vulnerability of Mistral was observed in almost all the attack families, most markedly that of multi-turn.

The defences transfer between models. Despite the significantly different behaviour of the two models in their unhardened form, the three-layer defence reduced attack success to 0% in both. This demonstrates that the defence mechanisms are not tailored to the peculiarities of a specific model, but retain their effectiveness regardless of it. A real cost is noted, however: on Mistral, which exhibits a greater tendency towards rejection, the hardened system recorded 90% benign success with 10% false refusals, against 100% and 0% respectively on Llama. The same set of defences is, consequently, slightly more conservative on Mistral - a real, measurable trade-off between security and usability, which a study limited to a single model would not have been in a position to reveal. The comparative evaluation is conducted by the file `cross_model.py`, through the command `python src/cross_model.py --models ollama:llama3.1:8b ollama:mistral`, with the results being stored in `results/cross_model.json`.

Execution Conditions and Parameters

All the numerical results of the present report derive from a single and fully specified experimental configuration, which is documented in detail so that its exact reproduction is possible. The parameters of the execution are the following:

- **Models and provider:** Llama 3.1 8B and Mistral 7B, executed locally via Ollama, without the use of a hosted API, access key or internet connection.
- **Decoding parameters:** temperature equal to zero, with every model call stored in transcripts files, so that the stable repetition of the responses is possible.
- **Date of execution:** June 2026.
- **Determinism regime:** with temperature 0, local decoding is in this instance essentially deterministic. Consequently, repeated executions of the ablation study yield identical results, that is, zero statistical dispersion. The absence of dispersion is declared as an expected consequence of the deterministic regime and as an indication of reproducibility, and not as a concealment of a lack of actual repetitions.

The entire execution chain - from the generation of the corpus and the executions of the baseline and of the hardened system, to the ablation study, the error analysis, the judge mechanism and the dashboard - is driven by the scripts that are documented in the accompanying file README.txt and is executed offline by default.

Limitations and threats to validity

In accordance with the principles of responsible reporting (lecture 7), the limits within which the findings hold are stated explicitly:

- **What is documented:** For the models Llama 3.1 8B and Mistral 7B, under temperature 0, the baseline leaked in 2/30 (Llama) and 8/30 (Mistral) attacks,

while the three-layer defence reduced this rate to 0/30, without substantial degradation of the service of benign requests, within the framework of the specific sixty-case benchmark.

- **Limited set of models:** The results concern two small, local open-source models. A larger or differently aligned (that is, trained differently with respect to safety) hosted model may behave differently; the examination of three or more models would further strengthen the generalisability of the conclusions.
- **Scope of the benchmark:** The sixty hand-written cases, although representative, cannot cover the totality of possible attacks. In particular, adaptive attackers are not yet included, that is, attacks that are dynamically reformulated after failure, adapting to the system's behaviour.
- **Fragility of the filters:** The first and third layers rely on pattern recognition and on the detection of canary. A disclosure phrased in paraphrase, which avoids both the phrases that trigger the defence and the exact form of the canary, could in principle escape. The judge mechanism, which evaluates semantically, mitigates but does not fully eliminate this risk.
- **Heuristic detection of refusals:** The recognition of a benign refusal is carried out by a heuristic (approximate) method. In borderline cases, evaluation by the judge or by human inspection would improve accuracy.

None of the above limitations negates the central finding of the work. On the contrary, they precisely delimit its scope of validity and indicate specific directions for future extension.

Conclusion

The present work developed a complete and reproducible framework for evaluating attacks and defences against prompt injection, applied to a realistic banking assistant based on a language model. Beyond the mere demonstration of isolated attacks, the work treated the phenomenon as a measurable security problem, implementing the full evaluation cycle: from the definition of the threat model and the construction of a structured benchmark, to the design of a multi-layer defence and its systematic assessment.

As regards **vulnerability**, the unhardened models, although they showed resilience against the bluntest attacks, proved vulnerable: the baseline leaked confidential data in 6.7% of the attacks for Llama 3.1 8B and in 26.7% for Mistral 7B, within a structured sixty-case benchmark. Of particular significance were two findings: the disclosure of the system prompt itself through a role-play attack, and the particular dangerousness of multi-turn attacks, which, especially on the weaker Mistral model, proved catastrophic (80% success on the unhardened system).

As regards **defence**, the proposed three-layer system reduced attack success to 0% on both models, while maintaining a high level of usability. A critical element of the evaluation was that leakage detection was carried out independently of the defence mechanisms, preventing any circularity and lending validity to the result. The ablation study demonstrated, in a measurable way, that the three layers are complementary and not redundant, since each blocks distinct categories of attacks; this finding transformed the principle of defence in depth from a general declaration into a documented conclusion.

Beyond the mandatory tasks, the two optional extensions further reinforced the findings: the independent judge confirmed the reliability of the leakage measurement, while the comparative evaluation between models revealed that, although vulnerability

depends strongly on the particular model, the effectiveness of the defences transfers between them - at the same time revealing a subtle security/usability trade-off that a single-model study would not have identified.

A principle running through the entire work was the use of unique canary tokens for the indisputable detection of leakage, the independent check through a second model, and the explicit statement both of what the evidence documents and of its limits. Overall, the work does not merely demonstrate that a defence system can stop prompt injection, but provides a rigorous, reproducible methodology for measuring this resilience - an element that also constitutes its most substantial contribution.

Deliverables and Execution Instructions

The submission is organised into a single folder project, with a clear separation between source code, data and documentation. The source code in the Python language is located in the folder `src/`, while the generated data are distributed across the folders `benchmark/` (the test corpus), `logs/` (the logs of the interactions), `results/` (the numerical results in JSON), `figures/` (the charts) and `transcripts/` (the stored responses for replay). The execution screenshots are located in the folder `screenshots/`. Full execution instructions are provided in the accompanying file `README.txt`. The following table maps each task of the work to the code files that implement it:

Task	Implemented by:
1. Assistant and threat model	<code>assistant_baseline.py</code> , <code>config.py</code> , <code>providers.py</code> , <code>logging_utils.py</code>
2. Benchmark	<code>benchmark_generator.py</code> , <code>canary.py</code>
3. Baseline evaluation	<code>runner.py</code> → <code>results/baseline_results.json</code>
4. Three defences	<code>defenses.py</code> , <code>assistant_defended.py</code>

Task	Implemented by:
5. Ablation	ablation.py → results/ablation.json
6. Error analysis	error_analysis.py → results/error_analysis.json
7. Explainability	dashboard.py
Advanced (1): Judge	judge.py → results/judge_cross_check.json
Advanced (2): Cross-model	cross_model.py → results/cross_model.json

Beyond the source code, the submission includes the present technical report, the documentation and instructions file (README.txt), the dependency list (requirements.txt), as well as the execution screenshots that document the reported results.